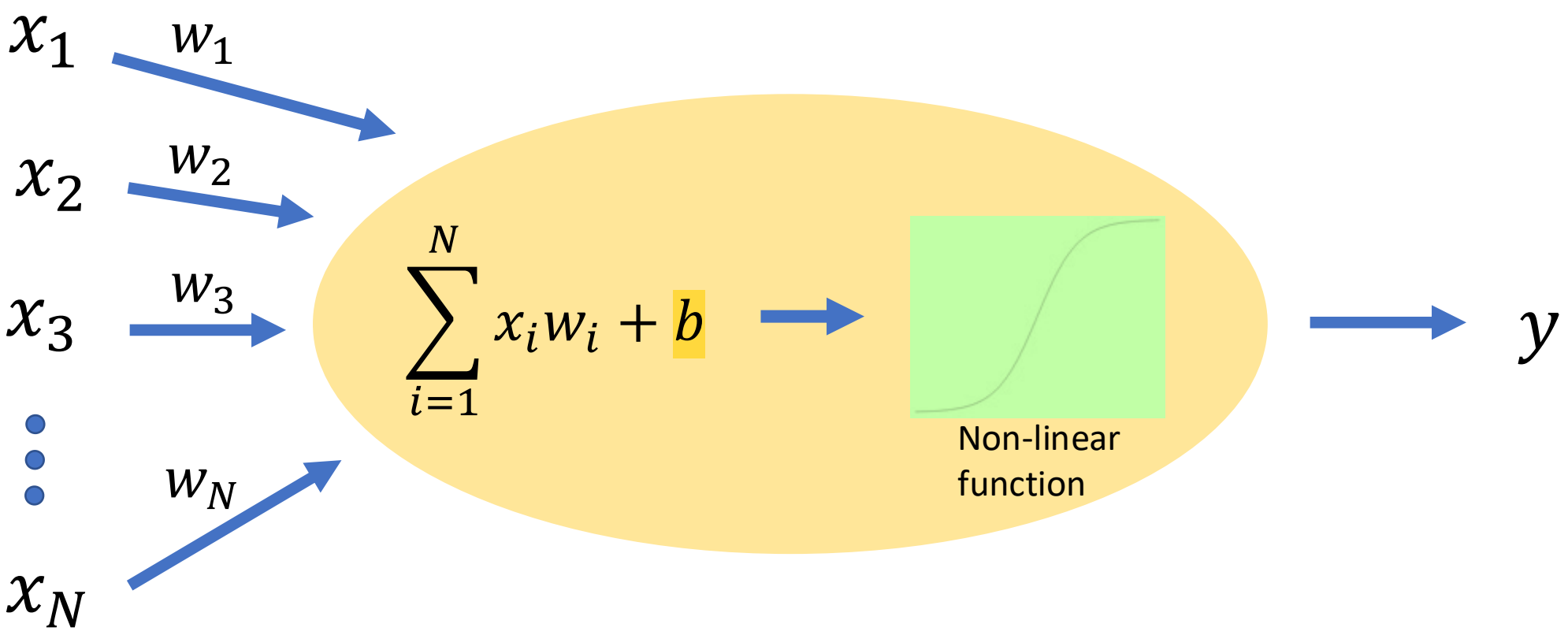
Neural Network / Deep Learning …

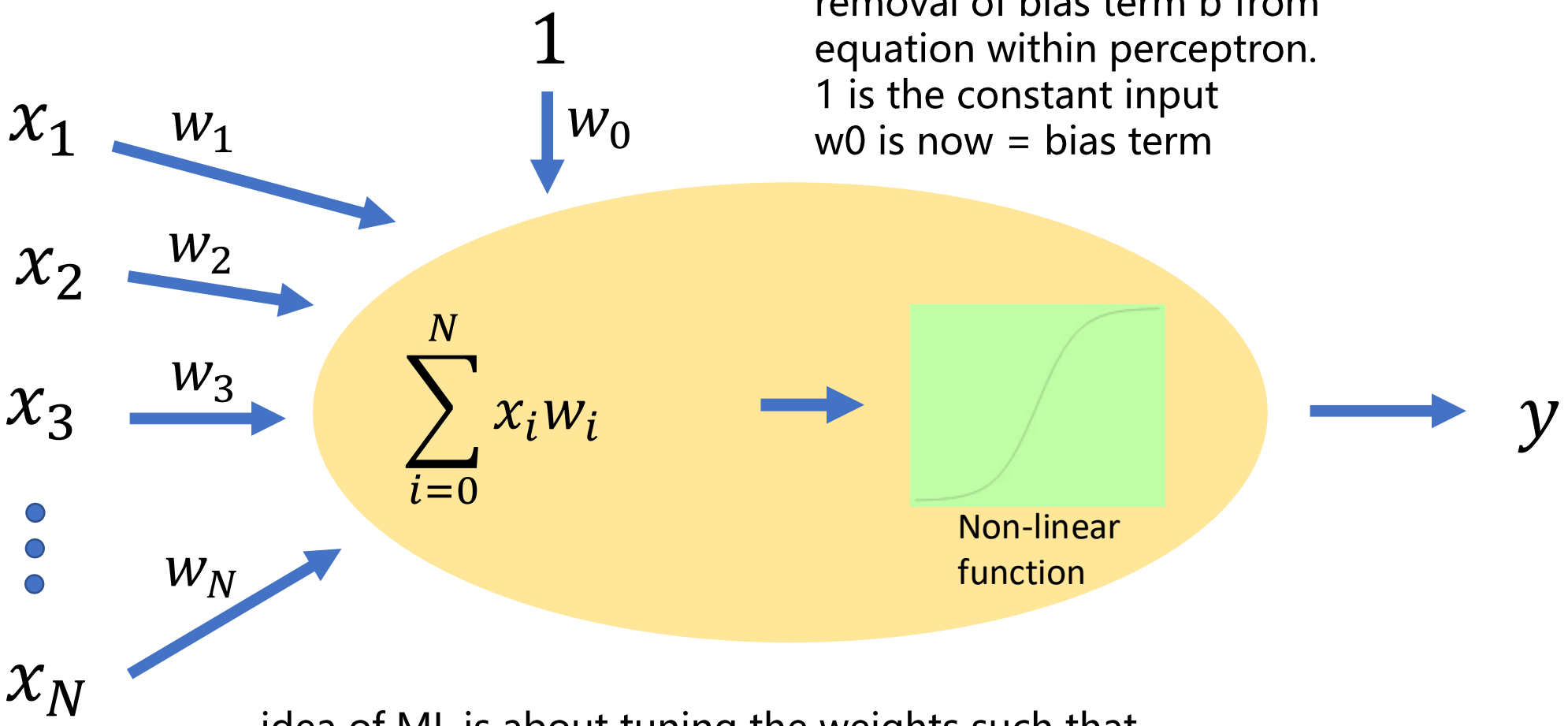
backward propagation initialised by a single api call in code
but need learn
reason being errors can happen
need understand how the backward propagation works behind the scene

Perceptron



Perceptron (another way to look at it)

removal of bias term b from equation within perceptron.
1 is the constant input
w0 is now = bias term

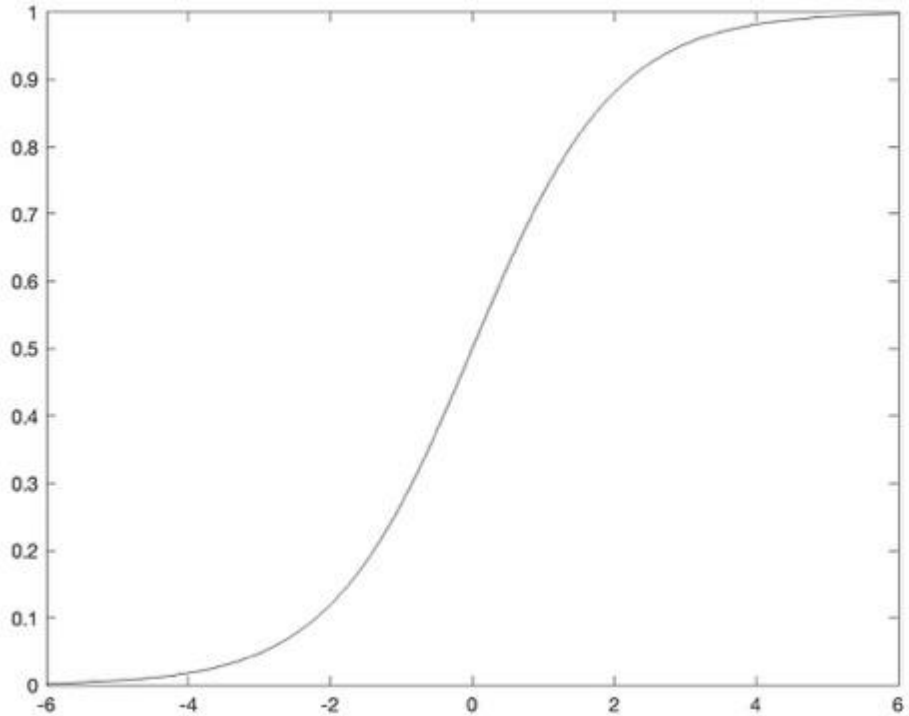


idea of ML is about tuning the weights such that the network is optimised for the task

Non-Linear Functions

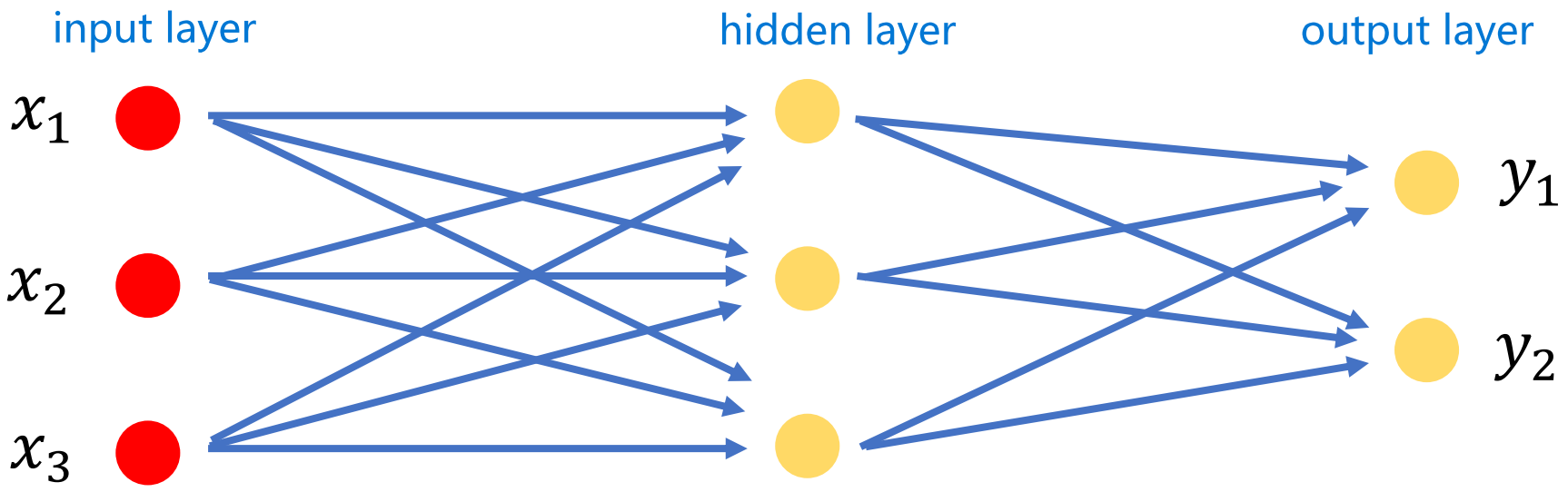
- Sigmoid function was a very popular choice in the earlier days of neural network

$$y = \frac{1}{1 + e^{-x}}$$



- Modern neural network uses a varied type of non-linear functions, eg. Relu

Neural Network:
connecting perceptrons to form a network

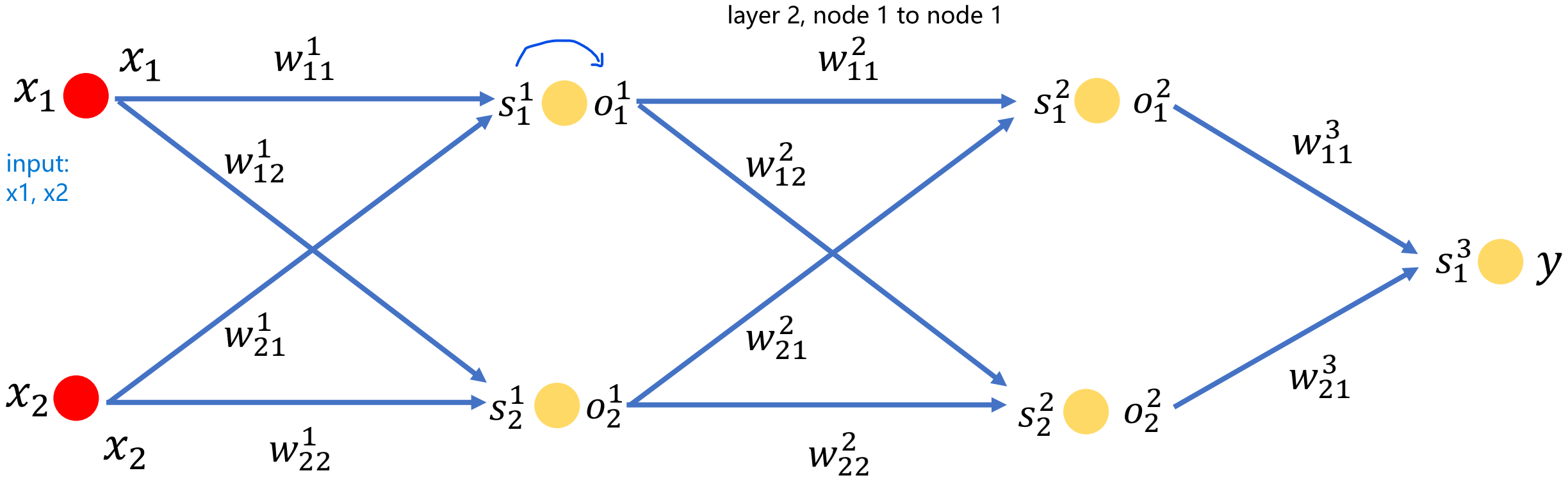


Input $x = [x_1 \quad x_2 \quad x_3]$

Output $y = [y_1 \quad y_2]$

Neural network supervised training process involves showing instances of input-output pairs to the network, and tune the weights (represented by blue arrows)

Training of a Neural Network using Backpropagation



$$s_i^l = \sum_{k=1}^{K^{l-1}} \left(o_k^{l-1} w_{ki}^l\right)$$

$$o_i^l = f\ \left(s_i^l\right)$$

$K^{l-1} = 2$ *in the above example*

f *is the non-linear function used eg. sigmoid*

l : current layer, l-1 : preceding layer, i : index of neuron in current layer, k : index of neuron in previo layer l-1

For an input data instance, $x = [x_1 \quad x_2]$

the network will produce an output y

Suppose the correct output (i.e. the target output) is t

then the error at the output is

$$E = (y - t)^2$$

Error = (predicted - true)^2

The goal of training is to make the error E as small as possible by tuning all the weights w_{ij}^{l}

i -> j : neuron ith to the jth neuron (refer above) of the next layer
l : current layer

We start from the output layer, where we need to tune

w_{ij}^3 where j = 1 since we only have one output node y

3 -> pre-activation layer to generate the final output

We need to find the partial derivative of E w.r.t. w_{i1}^3

how much E change when weight change

$$\frac{\partial E}{\partial w_{i1}^3} = \frac{\partial (y-t)^2}{\partial w_{i1}^3}$$

realise we cant solve this directly hence we use chain rule (refer later)

this is just calculus

For a sigmoid function, the derivative is simple

$$y = f(x) = \frac{1}{1 + e^{-x}}$$ Sigmoid function

$$\frac{\partial y}{\partial x} = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1+ e^{-x} -1}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} - \frac{1}{(1+e^{-x})^2} = y(1-y)$$

derivative of sigmoid

$$\frac{\partial E}{\partial w_{i1}^3} = \frac{\partial E}{\partial s_1^3}\frac{\partial s_1^3}{\partial w_{i1}^3} = \frac{\partial E}{\partial \cancel{y}}\frac{\partial \cancel{y}}{\partial \cancel{s_1^3}}\frac{\partial \cancel{s_1^3}}{\partial w_{i1}^3}$$

applying chain rule

E: error

w3i1: weight in third layer, from ith neuron to 1st neuron

s: weighted sum / net input / pre-activation

expand then solve

$$\frac{\partial E}{\partial y} = 2(y - t)$$

y: sigmoid output
s: input

$$\frac{\partial y}{\partial s_1^3} = y(1 - y)$$

$$\frac{\partial E}{\partial w_{i1}^3} = 2(y - t)y(1 - y)o_i^2$$

$$\frac{\partial s_1^3}{\partial w_{i1}^3} = o_i^2$$

s31: weighted sum of outputs from previous layer
s31 = w3i1 * o2i for all possible i (ith neuron from previous layer)
ds31 / dw3i1 = o21

Next, let's see how we tune the weights of the hidden layers

We need to find the partial derivative of E w.r.t. w_{ij}^{l}

$$\frac{\partial E}{\partial w_{ij}^{l}}$$

We will do so by continuing to apply the chain rule.

$$\frac{\partial E}{\partial w_{ij}^{l}} = \frac{\partial E}{\partial s_j^l}\frac{\partial s_j^l}{\partial w_{ij}^l} = \frac{\partial E}{\partial o_j^l}\frac{\partial o_j^l}{\partial s_j^l}\frac{\partial s_j^l}{\partial w_{ij}^l}$$

refer to above

$$\frac{\partial s_j^l}{\partial w_{ij}^l} = o_i^{l-1}$$

$$\frac{\partial E}{\partial o_j^l} = \sum_{k=1}^{K}\left(\frac{\partial E}{\partial s_k^{l+1}}\frac{\partial s_k^{l+1}}{\partial o_j^l}\right) = \sum_{k=1}^{K}\left(\frac{\partial E}{\partial o_k^{l+1}}\frac{\partial o_k^{l+1}}{\partial s_k^{l+1}}\frac{\partial s_k^{l+1}}{\partial o_j^l}\right)$$

$$= \sum_{k=1}^{K}\left(\underbrace{\frac{\partial E}{\partial o_k^{l+1}}}\, o_k^{l+1}\left(1 - o_k^{l+1}\right) w_{jk}^{l+1}\right)$$

$$\frac{\partial o_j^l}{\partial s_j^l} = o_j^l\left(1 - o_j^l\right)$$

This term had been computed earlier, remember, we compute backwards.
At the output layer, this term is simply

$$\frac{\partial E}{\partial o_k^{l+1}} = \frac{\partial E}{\partial y}$$

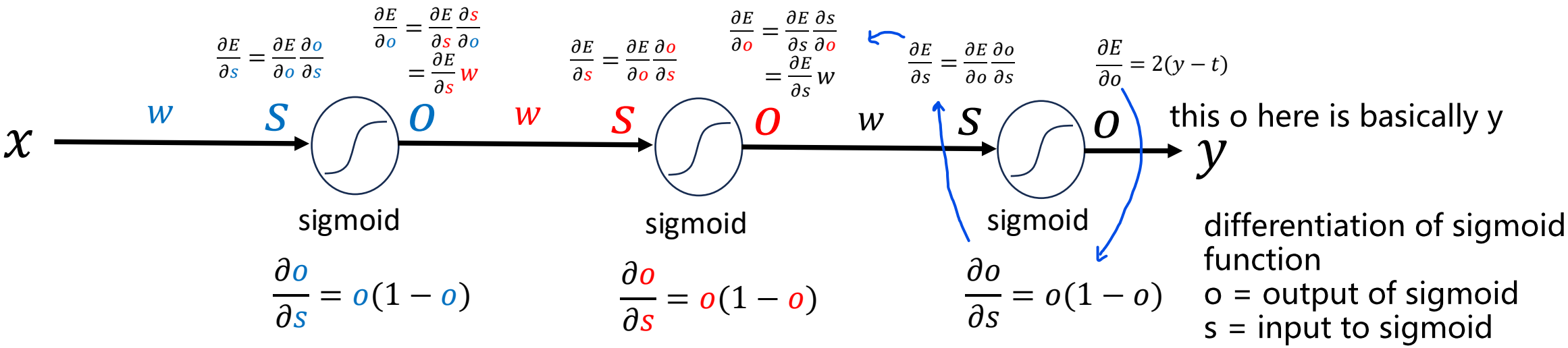
Now that the gradients w.r.t. each of the weights w_{ij}^{l} are found,

we can proceed to update the weights using

$$\Delta w_{ij}^{l} = -\eta \frac{\partial E}{\partial w_{ij}^{l}}$$

where η is known as the learning rate.

hyperparameter multiplied by the gradient to determine how much of the weight we should change.
gradient points the direction down the hill, learning rate is the length of that stride u take down the hill. if too large, u change the weights too fast, overshooting optimal



differentiation of sigmoid function
o = output of sigmoid
s = input to sigmoid

$$E = (y-t)^2$$

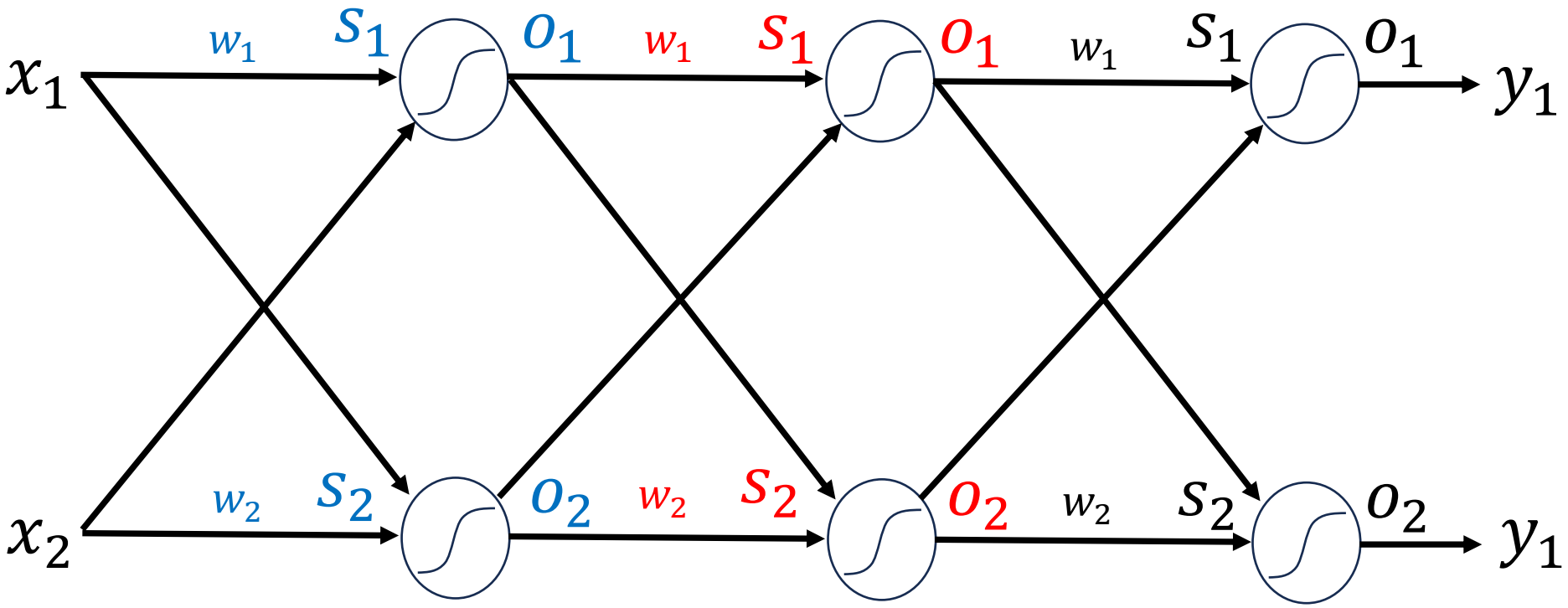
$$\frac{\partial E}{\partial y} = 2(y-t)$$

At output, $y = o$, therefore $\frac{\partial E}{\partial o} = 2(y-t)$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial s}\frac{\partial s}{\partial w} = \frac{\partial E}{\partial s}\, o$$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial s}\frac{\partial s}{\partial w} = \frac{\partial E}{\partial s}\, o$$

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial s}\frac{\partial s}{\partial w} = \frac{\partial E}{\partial s}\, x$$



$$\frac{\partial E}{\partial o_1} = \sum_{i=1}^{2} \frac{\partial E}{\partial s_i} \frac{\partial s_i}{\partial o_1}$$

```
class Net(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3,10)
        self.fc2 = nn.Linear(10,5)
        self.fc3 = nn.Linear(5, 1)
        self.double()

    def forward(self, x):
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        #x = F.relu(self.fc1(x))
        #x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Define the network architecture

1. Define the forward flow and the nonlinear activation function

Decide whether to run by CPU or by GPU

```
net = Net().to(device)
```

Decide what type of loss function to use

```
criterion = nn.MSELoss()
```

Decide learning rate and momentum parameters

```
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
```

```
for epoch in range(3):
    running_loss = 0.0
    for i, data in enumerate(train_dataloader):

        inputs = data['X']
        labels = data['y']

        # Forward pass
        outputs = net(inputs)
        outputs = torch.squeeze(outputs,2)

        loss = criterion(outputs, labels)

        # Zero the parameter gradients
        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

        if i % 10 == 1:
            print('[%d, %5d] loss: %.3f' %
                (epoch + 1, i, running_loss / 10))
            running_loss = 0.0
```

backpropagation